



STUDIO H5

CNPJ 51.965.304/0001-24

DOCUMENTAÇÃO TÉCNICA

Sistema **Vidroforte SAC** Guia completo para manutenção

Documento técnico detalhado com todas as informações necessárias para compreender, executar, manter e evoluir o sistema. Direcionado ao desenvolvedor(a) responsável pela continuidade da manutenção após a entrega feita pela Studio H5.

Sumário

1. Visão geral do sistema	03
2. Stack tecnológica	03
3. Arquitetura	04
4. Estrutura de diretórios	04
5. Como executar em ambiente local	05
6. Banco de dados — principais entidades	06
7. Endpoints principais	07
8. Regras de negócio críticas	08
9. Sistema de permissões	09
10. Uploads e arquivos anexos	10
11. Envio de e-mail (MailerSend)	10
12. Deploy e ambiente de produção	11
13. Boas práticas e convenções	11
14. Contato e suporte	12

Sobre este documento: foi escrito por Gabriel Souza (Studio H5), autor do sistema, com o objetivo de dar autonomia total ao desenvolvedor que for assumir a manutenção. Todos os fluxos, decisões arquiteturais e pontos delicados estão descritos aqui. Em caso de dúvida sobre trechos específicos do código, entre em contato — meu contato está na última seção.



1 Visão geral do sistema

O **Sistema Vidroforte SAC** é uma plataforma composta por dois ambientes integrados que compartilham o mesmo banco de dados:

Portal do Cliente **PÚBLICO**

Ambiente acessado pelo consumidor final que comprou produtos Vidroforte. Permite abrir ocorrências (reclamações/solicitações de garantia), acompanhar o andamento com timeline, responder solicitações de documentos, escolher opções de garantia quando aprovada, e solicitar reanálise em caso de recusa.

Painel Administrativo **INTERNO**

Ambiente utilizado pela equipe da Vidroforte para tratar as ocorrências recebidas. Contempla dashboard com KPIs, gestão de ocorrências, RNCs internas (Registro de Não Conformidade), custos, relatórios com exportação em Excel, gestão de usuários por setor, sistema de permissões granular e controle de prazos.

Cada ambiente possui seu próprio backend (API REST) e seu próprio frontend (SPA leve em HTML/CSS/JavaScript puro). Ambos os backends conectam ao mesmo banco de dados para consistência de informações.

2 Stack tecnológica

Backend

Tecnologia	Versão	Papel
Java	21 (LTS)	Linguagem principal
Spring Boot	3.5.0	Framework web e IoC
Spring Security	6.5.0	Autenticação e autorização
Spring Data JPA	3.5.0	Acesso ao banco (via Hibernate)
Hibernate ORM	6.6.15	Mapeamento objeto-relacional
Flyway	11.7.2	Versionamento e migração do schema
MySQL Connector	9.2.0	Driver do banco em produção
H2 Database	2.3.232	Banco embarcado para desenvolvimento
Apache POI	5.4.0	Geração de arquivos Excel (relatórios)
JWT	0.12.6	Geração e validação de tokens JWT
Lombok	latest	Redução de boilerplate em entities/DTOs
Maven	3.9+	Gerenciamento de dependências e build

Frontend

Tecnologia	Papel
HTML5	Estrutura das telas
CSS3 (puro)	Estilização — Grid, Flexbox, variáveis CSS, responsividade nativa
JavaScript (ES6+)	Lógica de interface, chamadas fetch, manipulação de DOM. Sem framework.

Importante: o sistema não utiliza React, Vue, Angular, TypeScript ou bundlers. A escolha por JavaScript vanilla foi deliberada — mantém o sistema leve, sem build step, fácil de deployar e sem cadeia de dependências frágeis.

3 Arquitetura

Quatro projetos independentes que se comunicam via HTTP e compartilham o banco de dados:

Projeto	Porta	Repositório local
cliente-api — backend do portal público	8081	vidroforte-SAC-api
admin-api — backend do painel interno	8082	admin-vidroforte-SAC-api
cliente-frontend — SPA do portal público	5501 (dev)	vidroforte-SAC
admin-frontend — SPA do painel interno	5502 (dev)	admin-vidroforte-SAC

Como se comunicam

- ✓ **Cliente-frontend** → **cliente-api**: portal público consome sua própria API
- ✓ **Admin-frontend** → **admin-api**: painel interno consome sua própria API
- ✓ **Admin-frontend** → **cliente-api (arquivos)**: apenas para baixar anexos que o cliente enviou (via `/uploads/**`)
- ✓ **Cliente-frontend** → **admin-api (arquivos)**: apenas para baixar anexos de garantia enviados pelo admin
- ✓ **cliente-api** ↔ **admin-api**: **não se comunicam diretamente**. Consistência é feita via banco compartilhado

Entidades espelho

Como ambos os backends acessam as mesmas tabelas, existem **entidades espelho** em cada backend representando as tabelas do outro. Exemplo: `Cliente` existe no cliente-api (com toda a lógica) e também no admin-api (versão simplificada, somente-leitura para exibição).

Ponto de atenção: ao criar novos campos em uma entidade que aparece nos dois lados, é necessário replicar a coluna nos dois arquivos `.java`. Caso contrário, o Hibernate vai ignorar o campo do lado que não o mapeou.

4 Estrutura de diretórios

Estrutura padrão de cada backend Spring Boot:

```
src/main/java/com/vidroforte/{sac|admin}/
├── config/           Configurações (Security, Async, Bootstrap, CORS)
├── controller/       Endpoints REST (recebem requisições HTTP)
├── dto/              Records Java com contratos de entrada e saída
├── model/            Entities JPA e enums
├── repository/       Interfaces Spring Data JPA
├── security/         JwtAuthFilter, JwtService, AuthenticatedX
└── service/          Regras de negócio (transacional)

src/main/resources/
├── application.properties      Configuração base (produção)
├── application-dev.properties  Configuração de desenvolvimento
├── application-test.properties Configuração para testes
└── db/migration/              Migrações Flyway (V1__..., V2__...)
```

Cada frontend possui estrutura semelhante:

```
{cliente|admin}-frontend/
├── assets/
│   ├── css/      Estilos por escopo (app.css, admin.css)
│   ├── js/       Módulos JS por página + api.js (client base)
│   └── img/       Logos e imagens estáticas
└── index.html     Home
```


5 Como executar em ambiente local

Pré-requisitos

- ✓ Java 21 instalado (recomendado via SDKMAN ou OpenJDK oficial)
- ✓ Maven 3.9+ (ou usar o wrapper `./mvnw` incluído no projeto)
- ✓ Node não é necessário — o frontend é servido estaticamente
- ✓ MySQL não é necessário em dev — o sistema usa H2 embarcado em modo arquivo

Ordem de execução

Os backends compartilham o banco H2. Para evitar conflito de criação de schema, siga esta ordem:

1. Limpar banco de desenvolvimento (opcional, primeira execução)

```
lsof -ti :8081 :8082 | xargs -r kill; sleep 2
rm -f /tmp/vidroforte_sac_dev.*
```

2. Iniciar o cliente-api (Terminal 1)

```
cd vidroforte-SAC-api
./mvnw spring-boot:run -Dspring-boot.run.profiles=dev
```

Aguarde a mensagem `Started VidroforteSacApiApplication`. A API estará disponível em `http://localhost:8081`.

3. Iniciar o admin-api (Terminal 2)

```
cd admin-vidroforte-SAC-api
./mvnw spring-boot:run -Dspring-boot.run.profiles=dev
```

O admin cria automaticamente um usuário administrador na primeira execução, com credenciais definidas em `application-dev.properties`:

E-MAIL

admin@vidroforte.com.br

SENHA INICIAL

MudarNoLogin@2026

4. Servir os frontends

Frontends são pastas de arquivos estáticos. Basta abrir com qualquer servidor HTTP local — recomendado usar a extensão **Live Server** do VS Code, ou:

```
cd vidroforte-SAC && python3 -m http.server 5501
cd admin-vidroforte-SAC && python3 -m http.server 5502
```

Acesse:

- ✓ Portal do cliente: `http://localhost:5501`
- ✓ Painel admin: `http://localhost:5502`

Ponto importante: se as portas 8081 ou 8082 ficarem travadas após um crash, mate com `lsof -ti :8081 :8082 | xargs -r kill`. Nunca inicie os dois backends em paralelo sem esperar o primeiro terminar de subir — o H2 file lock pode causar conflito.

6 Banco de dados — principais entidades

Tabelas centrais que sustentam toda a operação do sistema. A lista completa está no código.

Tabela	Papel
<code>clientes</code>	Cadastro do consumidor que abre ocorrências no portal público
<code>ocorrencias</code>	Tabela central — cada linha é uma reclamação/solicitação de garantia
<code>ocorrencia_anexos</code>	Fotos, notas fiscais e PDFs anexados pelo cliente
<code>ocorrencia_historico</code>	Timeline visível ao cliente (mudanças de status feitas no portal)
<code>ocorrencia_acao</code>	Timeline completa vista pelo admin (comentários internos e públicos, encaminhamentos, mudanças)
<code>ocorrencia_atribuicao</code>	Setor e responsável atual de cada ocorrência
<code>ocorrencia_custos</code>	Custos lançados pela equipe interna atrelados a cada ocorrência
<code>ocorrencia_garantia_anexos</code>	Nota fiscal, recibo de garantia — anexados pelo admin, visíveis ao cliente no modal de aprovação
<code>exigencia_arquivo</code>	Solicitação de novos arquivos ao cliente (quando NF é ilegível, por exemplo)
<code>rnc_internas</code>	Registros de Não Conformidade — fluxo interno independente da ocorrência do cliente
<code>rnc_evidencias</code>	Fotos, desenhos e atas anexados às RNCs
<code>rnc_historico</code>	Timeline completa das movimentações da RNC
<code>rnc_custos</code>	Custos internos gerados pela RNC (viagens, retrabalho, descarte)
<code>usuarios</code>	Funcionários da Vidroforte com acesso ao painel admin
<code>setor_fixo_alias</code>	Aliases customizáveis para renomear setores padrão sem alterar o enum
<code>setor_customs</code>	Setores criados dinamicamente pela Vidroforte além dos padrão

Estratégia de schema

Em **desenvolvimento**, o Hibernate cria/atualiza o schema automaticamente (`ddl-auto=update`). Em **produção**, o schema é gerenciado pelo **Flyway** com scripts versionados em `src/main/resources/db/migration/`. Toda alteração de estrutura em produção deve ser feita via nova migration (nunca alterando manualmente).

Regras conhecidas do MySQL Railway: não suporta `ALTER TABLE ADD COLUMN IF NOT EXISTS`. Ao criar migrations que adicionam colunas, use a sintaxe simples `ALTER TABLE x ADD COLUMN y ...` — se já existir, o Flyway acusa erro e você precisa marcar a migration como aplicada manualmente.

7 Endpoints principais

Lista resumida dos endpoints mais importantes. A lista completa está no código dos `Controller.java`.

cliente-api (porta 8081)

Método	Path	Descrição
POST	/api/auth/registrar	Registra novo cliente
POST	/api/auth/login	Login e emissão de JWT
POST	/api/auth/esqueci-senha	Solicita link de recuperação por e-mail
POST	/api/ocorrencias	Abre nova ocorrência
POST	/api/ocorrencias/{id}/anexos	Sobe anexo (multipart)
GET	/api/ocorrencias/{protocolo}	Detalhe completo da ocorrência
POST	/api/ocorrencias/{p}/replica	Solicita reanálise após garantia recusada
POST	/api/ocorrencias/{p}/garantia/escolher	Cliente escolhe opção A ou B
POST	/api/ocorrencias/{id}/exigencia/responder	Envia arquivo pedido pelo admin

admin-api (porta 8082) — principais grupos

Método	Path	Descrição
POST	/api/admin/auth/login	Login do usuário admin
GET	/api/admin/dashboard/resumo	KPIs e gráficos do dashboard
GET	/api/admin/dashboard/ocorrencias-expiradas	Ocorrências com prazo estourado
GET	/api/admin/dashboard/rnc-kpis	KPIs das RNCs em tratativa
GET	/api/admin/ocorrencias	Lista paginada com filtros
GET	/api/admin/ocorrencias/{p}	Detalhe completo
POST	/api/admin/ocorrencias/{p}/status	Altera status (com validação de transição)
POST	/api/admin/ocorrencias/{p}/encaminhar	Encaminha para outro setor
POST	/api/admin/ocorrencias/{p}/garantia/anexos	Anexa nota fiscal / recibo à garantia aprovada
GET/POST/DELETE	/api/admin/ocorrencias/{p}/custos	CRUD de custos
POST	/api/admin/rnc	Abre RNC interna
PUT	/api/admin/rnc/{id}/prazo-devolutiva	Define prazo de devolutiva
POST	/api/admin/rnc/{id}/evidencias	Sobe evidência (foto/desenho/ata)
GET	/api/admin/relatorios	Relatório agregado com filtros
GET	/api/admin/relatorios/exportar.xlsx	Exporta relatório em Excel

8 Regras de negócio críticas

Estas regras estão implementadas no backend (fonte da verdade) e replicadas no frontend para melhor experiência do usuário. Alterá-las em um lado exige alterar no outro.

Transição de status da ocorrência

- ✓ Status **ENCERRADO** é terminal — nenhum status pode ser alterado após
- ✓ Ocorrência encerrada bloqueia: alterar status, encaminhar, lançar custos
- ✓ Ocorrência encerrada permite: comentar, abrir RNC, imprimir
- ✓ Não é permitido regredir de **GARANTIA_APROVADA** para **GARANTIA_RECUSADA**
- ✓ Não é permitido voltar de **GARANTIA_APROVADA** para **DOCUMENTACAO_PENDENTE**
- ✓ Reanálise permitida: **GARANTIA_RECUSADA** → **GARANTIA_APROVADA**
- ✓ Doc. pendente aceito de qualquer status (exceto após aprovada)
- ✓ Fora dessas exceções, só avança na ordem canônica do fluxo

Réplica / Reanálise

Após garantia recusada, o cliente tem direito a solicitar **1 única reanálise**. Após a segunda decisão do admin, a ocorrência fica bloqueada — nenhuma alteração de status é permitida, apenas abertura de RNC interna.

Aprovação de garantia

Quando o admin aprova a garantia, obrigatoriamente preenche: detalhes que aparecem para o cliente, opção A e opção B. Um modal obrigatório aparece para o cliente na próxima vez que ele acessar a ocorrência — ele precisa escolher A ou B para prosseguir. A escolha fica registrada e visível no histórico.

Prazo de tratativa

Toda ocorrência criada pelo cliente tem um prazo automático calculado em **horas úteis** (config `app.prazo.tratativa.horas-uteis`, padrão 48h). Horas úteis contam 24h/dia mas pulam sábado e domingo. Se o prazo expira e a ocorrência ainda está em **RECEBIDO** ou **DOCUMENTACAO_PENDENTE**, ela aparece como atrasada no dashboard e recebe destaque vermelho na lista.

Prazo de devolutiva das RNCs

Ao ser designado como responsável de uma RNC, o usuário é obrigado a definir prazo de devolutiva antes de qualquer outra ação (modal obrigatório abre no primeiro acesso). Exceção: quando a RNC retorna para o próprio criador, não é solicitado novo prazo.

Roteamento automático para Qualidade

Ao clicar em "Enviar para validação da Qualidade" numa RNC, o sistema busca automaticamente um usuário ativo do setor **QUALIDADE** com menor carga de RNCs abertas e atribui a ele. Se não houver nenhum usuário Qualidade cadastrado, retorna erro claro pedindo cadastro.

Encerramento por RNC final

Custos podem ser lançados na RNC (não só na ocorrência). O resumo financeiro do detalhe mostra: custo da RNC + custo da ocorrência vinculada + total. Ambos alimentam o dashboard e os relatórios de custo.

9 Sistema de permissões

Estrutura

- ✓ Todo usuário admin tem um **Setor** (POS_VENDAS, PRODUCAO, LOGISTICA, COMERCIAL, MANUTENCAO, QUALIDADE, ADMIN_DONO)
- ✓ Usuários do setor **ADMIN_DONO** têm automaticamente **todas as permissões** (super-usuário)
- ✓ Demais usuários têm permissões individuais atribuídas via enum **Permissao**

Como funciona no backend

No login, o `JwtService` gera o token JWT contendo **id**, **email**, **setor** e a **lista de permissões**. A cada requisição, o `JwtAuthFilter` valida o token e popula o `AuthenticatedAdmin` no `SecurityContext`. Endpoints protegidos usam `@PreAuthorize("hasAuthority('PERM_X')")` — o filtro converte a lista de permissões em `SimpleGrantedAuthority` com prefixo `PERM_`.

Ponto importante: o filtro concede automaticamente todas as permissões `PERM_*` quando o setor é `ADMIN_DONO`. Isso evita a necessidade de repetir `@PreAuthorize` com múltiplas condições.

Como funciona no frontend

O `api.js` guarda a sessão em `localStorage` (token, id, nome, email, setor, permissões). A função `VFA.hasPermission(perm)` é usada em todos os pontos de renderização para esconder/mostrar botões, itens de menu e blocos inteiros conforme o usuário logado.

Auto-lock de admin

O sistema impede que o último usuário **ADMIN_DONO** seja rebaixado para outro setor (auto-lockout protection). Além disso, o bootstrap em cada boot promove automaticamente o e-mail `admin@vidroforte.com.br` para **ADMIN_DONO** caso ele tenha sido movido acidentalmente.

Setores customizáveis

Além dos 7 setores fixos no enum, o admin pode criar **setores customizados** via aba Setores. Também pode renomear os setores fixos usando aliases (tabela `setor_fixo_alias`) sem alterar código.

10 Uploads e arquivos anexos

Arquivos são armazenados em disco local em cada backend:

- ✓ **cliente-api**: anexos que o cliente sobe (foto do dano, NF, etiqueta, logo, vídeo) — em `uploads/ocorrencias/{ocId}/`
- ✓ **admin-api**: anexos que o admin sobe (evidências de RNC, anexos de garantia) — em `uploads/rnc/{id}/` e `uploads/garantia/{ocId}/`

Cada backend serve seus próprios arquivos via `/uploads/**` (endpoint público). O caminho é gerado pelo `StorageService` com UUID + extensão original.

Recomendação para produção: em produção, migrar o armazenamento para **Cloudinary** ou **Amazon S3**. O código já foi escrito com interface abstrata (`StorageService`) — trocar a implementação exige apenas uma classe nova. Manter arquivos em disco em produção limita escalabilidade horizontal e complica backup.

11 Envio de e-mail (MailerSend)

O sistema utiliza o serviço **MailerSend** para todos os envios de e-mail transacional. Ver documento separado "Justificativa Técnica — MailerSend" para o contexto de contratação.

Configuração

A API Key do MailerSend é configurada em `application-prod.properties` (ou variável de ambiente `MAILERSEND_API_KEY`). O remetente padrão é definido em `app.mail.from` .

Arquitetura de envio

Envios são disparados **assincronamente** via `@Async` e enfileirados em uma tabela `email_outbox` . Um job scheduler processa a fila e tenta reenviar em caso de falha (retry exponencial, máximo 5 tentativas). Isso garante que:

- ✓ Requisições HTTP não bloqueiam esperando o SMTP responder
- ✓ Se o provider cair, e-mails ficam persistidos e são enviados quando voltar
- ✓ Toda tentativa de envio é auditada (útil para reclamações do tipo "não recebi")

Bounces e reputação

O MailerSend envia webhook em caso de bounce (e-mail inválido). O endpoint `/api/webhooks/mailersend` processa e marca `cliente.emailValido = false` — o admin vê alerta vermelho no perfil do cliente pedindo contato por telefone.

12 Deploy e ambiente de produção

Perfis Spring

Três perfis configurados: `dev` (H2 file), `prod` (MySQL) e `test` (H2 em memória para testes automatizados).

Recomendação de infraestrutura

Componente	Sugestão
Hospedagem backend	Railway, Render ou VPS Locaweb com Java 21
Banco de dados	MySQL 8 gerenciado (Railway, PlanetScale ou RDS)
Hospedagem frontend	Vercel, Netlify ou mesma VPS servindo via nginx
Storage de arquivos	Cloudinary (recomendado) ou AWS S3
Envio de e-mail	MailerSend Iniciante (50k/mês)
Monitoramento	UptimeRobot (grátis) para checar disponibilidade

Build de produção

```
./mvnw clean package -DskipTests
java -jar target/vidroforte-sac-api-0.0.1-SNAPSHOT.jar --spring.profiles.active=prod
```

Variáveis de ambiente esperadas em produção

- ✓ `SPRING_DATASOURCE_URL` — JDBC do MySQL
- ✓ `SPRING_DATASOURCE_USERNAME` e `PASSWORD`
- ✓ `APP_JWT_SECRET` — chave HMAC de 256 bits (nunca commitar)
- ✓ `MAILERSEND_API_KEY` — API key do provider
- ✓ `APP_UPLOAD_DIR` — caminho absoluto para armazenamento de arquivos
- ✓ `APP_FRONTEND_URL` — URL pública do frontend para links de e-mail

13 Boas práticas e convenções

- ✓ **Nunca commitar segredos** — todos os secrets ficam em `application-prod.properties` (não versionado) ou variáveis de ambiente
- ✓ **DTOs sempre em records** — imutáveis e sem risco de vazarem entidades
- ✓ **@Transactional(readOnly = true)** em qualquer método que retorne DTO com dados de relacionamento LAZY — evita `LazyInitializationException`
- ✓ **Registrar toda ação em histórico** — mudanças de status, encaminhamentos, custos, evidências e prazos ficam em tabelas de histórico dedicadas
- ✓ **Frontend sem estado global compartilhado** — cada página tem seu próprio JS, sem dependências entre arquivos além de `api.js` e `shell.js`
- ✓ **CSS por escopo** — evitar classes utilitárias globais que quebram fácil; preferir classes com prefixo semântico (`mst-`, `gar-`, `exp-` ...)
- ✓ **Migrações Flyway são imutáveis** — nunca alterar arquivo `V*.sql` após aplicado em produção. Criar `V{n+1}__correcao_X.sql`

14 Contato e suporte

Fico à disposição para esclarecer dúvidas técnicas, discutir decisões de arquitetura ou apoiar em pontos específicos durante a transição:

Gabriel Souza — Autor do Sistema

EMPRESA

Studio H5

E-MAIL

contato@studioh5.com.br

CNPJ

51.965.304/0001-24

WHATSAPP

(27) 9 9783-1582

Reunião técnica de transição: recomendo agendarmos uma call de aproximadamente 1 hora para eu apresentar pessoalmente a estrutura do código, tirar dúvidas e mostrar os fluxos mais delicados na prática. Isso acelera muito a curva de aprendizado.

Modelo de manutenção sugerido

Existem duas modalidades de contrato de manutenção mensal disponíveis com a Studio H5, que podem ser adotadas em paralelo à manutenção interna, dando cobertura adicional em casos de picos, dúvidas complexas ou implementações que envolvam a arquitetura original do sistema. Os documentos comerciais podem ser solicitados a qualquer momento.

Encerramento

Foi uma satisfação desenvolver este sistema. Todo o cuidado técnico foi tomado para deixá-lo simples de entender, sem dependências obscuras e com fluxos claros. Boa sorte na continuidade do projeto.

Gabriel Souza

Studio H5 · Desenvolvimento de Sistemas